

TARANOVA'S PIZZA

SQL Sales Analysis

Business Requirements & Query Documentation

Prepared by: Business Analytics Team

Database: taranovas_pizza

Document Type: SQL Query & Business Insight Log

This document catalogs thirteen SQL-based business questions used to analyze order volume, revenue, product mix, and category performance for Taranova's Pizza. Each entry pairs a stated business requirement with its supporting query, result output, and a data-grounded business insight.

Question 1

Retrieve the Total Number of Orders Placed

Business Requirement

The restaurant management wants to determine the total number of customer orders placed during the analysis period. This metric helps measure overall business activity and provides an understanding of customer demand.

Tables Used

Table Name
orders

SQL Concepts Used

SQL Concept

COUNT()

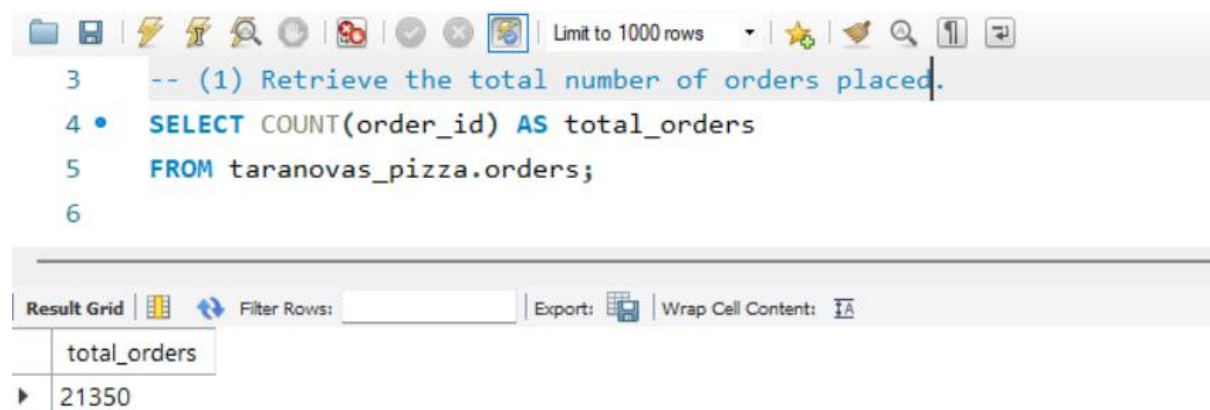
Aggregate Function

Column Alias (AS)

SQL Query

```
SELECT COUNT(order_id) AS total_orders
FROM taranovas_pizza.orders;
```

Output



The screenshot shows a SQL query editor interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query text is as follows:

```
3  -- (1) Retrieve the total number of orders placed.
4  •  SELECT COUNT(order_id) AS total_orders
5     FROM taranovas_pizza.orders;
6
```

Below the editor is the 'Result Grid' section. It features a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The grid displays a single column named 'total_orders' with a value of 21350.

total_orders
21350

Business Insight

The query returns 21,350 total orders for the analysis period. This figure establishes the baseline transaction volume for the business and is the denominator management should use whenever computing per-order metrics — such as average order value or items per order — to track whether growth is coming from more customers or from larger baskets.

Question 2

Calculate the Total Revenue Generated from Pizza Sales

Business Requirement

The management wants to calculate the total revenue generated from all pizza sales during the analysis period. This metric is essential for evaluating the restaurant's overall financial performance and profitability.

Tables Used

Table Name
order_details
pizzas

SQL Concepts Used

SQL Concept
SUM()
INNER JOIN
Arithmetic Calculation (Quantity x Price)
ROUND()
Column Alias (AS)

SQL Query

<pre>SELECT ROUND(SUM(od.quantity * p.price), 2) AS total_sales FROM taranovas_pizza.order_details AS od JOIN taranovas_pizza.pizzas AS p ON od.pizza_id = p.pizza_id;</pre>
--

Output

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
-- (2) Calculate the total revenue generated from pizza sales.
SELECT
  ROUND(SUM(od.quantity * p.price), 2) AS total_sales
FROM
  taranovas_pizza.order_details AS od
JOIN
  taranovas_pizza.pizzas AS p
ON
  od.pizza_id = p.pizza_id;

-- (3) Identify the highest-priced pizza
USE taranovas_pizza;
SELECT
  pizza_types.name AS pizza_name,
  pizzas.price
```

The result grid shows the following data:

total_sales
817860.05

Below the result grid, there is a tab labeled "Result 5" and an "Output" section.

Business Insight

Total revenue for the period comes to \$817,860.05. Set against the 21,350 orders confirmed in Question 1, this puts average order value at approximately \$38.31 — a concrete benchmark management can use to evaluate the revenue impact of future menu price changes, combo offers, or upsell initiatives.

Question 3

Identify the Highest-Priced Pizza

Business Requirement

The management wants to identify the most expensive pizza available on the menu. This helps in analyzing premium products and evaluating pricing strategies for high-value menu items.

Tables Used

Table Name
pizza_types
pizzas

SQL Concepts Used

SQL Concept
INNER JOIN
ORDER BY
LIMIT
Column Alias (AS)

SQL Query

```
SELECT
    pizza_types.name AS pizza_name,
    pizzas.price
FROM
    pizza_types
INNER JOIN
    pizzas
ON
    pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY
    pizzas.price DESC
LIMIT 1;
```

Output

The screenshot shows a SQL IDE interface. On the left, a 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'order_details', 'pizzas', and 'pizza_types' under the 'taranovas_pizza' schema. The main editor window displays a SQL query across multiple lines, with line numbers 16 through 30. The query identifies the highest-priced pizza by joining 'pizza_types' and 'pizzas' tables, ordering by price in descending order, and limiting the result to one row. Below the query editor, a 'Result Grid' pane shows the output of the query, which is a single row: 'The Greek Pizza' with a price of 35.95.

```
16      od.pizza_id = p.pizza_id;
17
18      -- (3) Identify the highest-priced pizza
19 •    USE taranovas_pizza;
20 •    SELECT
21      pizza_types.name AS pizza_name,
22      pizzas.price
23 FROM
24      pizza_types
25 INNER JOIN
26      pizzas
27      ON pizza_types.pizza_type_id = pizzas.pizza_type_id
28 ORDER BY
29      pizzas.price DESC
30 LIMIT 1;
```

pizza_name	price
The Greek Pizza	35.95

Business Insight

The Greek Pizza is the single highest-priced item on the menu at \$35.95. Because this price point comes from a specialty recipe rather than an oversized XL/XXL variant, it functions as the natural ceiling of the pricing ladder and is a strong candidate for premium placement on the menu and for anchoring value comparisons against lower-priced items.

Question 4

Identify the Most Common Pizza Size Ordered

Business Requirement

The restaurant management wants to determine which pizza size is ordered most frequently. Understanding customer preferences for pizza sizes helps improve inventory planning and production efficiency.

Tables Used

Table Name
order_details
pizzas

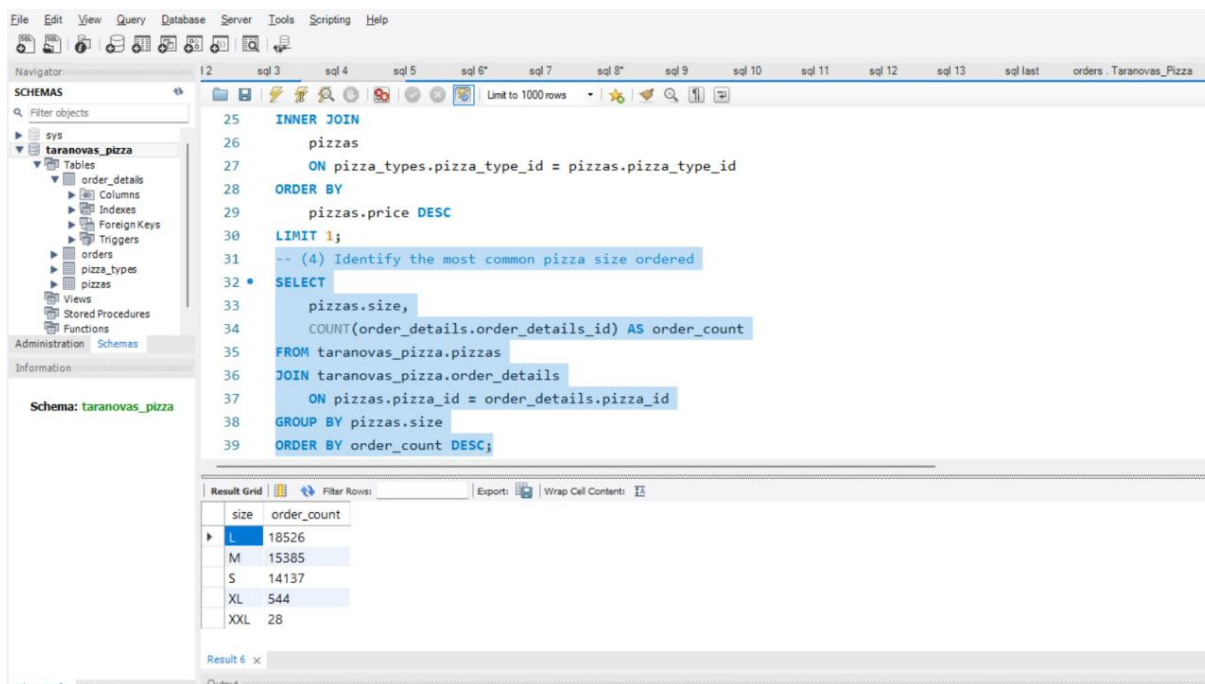
SQL Concepts Used

SQL Concept
COUNT()
GROUP BY
ORDER BY
INNER JOIN
Column Alias (AS)

SQL Query

```
SELECT
    pizzas.size,
    COUNT(order_details.order_details_id) AS order_count
FROM
    taranovas_pizza.pizzas
JOIN
    taranovas_pizza.order_details
ON
    pizzas.pizza_id = order_details.pizza_id
GROUP BY
    pizzas.size
ORDER BY
    order_count DESC;
```

Output



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor displays the following SQL code:

```
25 INNER JOIN
26     pizzas
27     ON pizza_types.pizza_type_id = pizzas.pizza_type_id
28 ORDER BY
29     pizzas.price DESC
30 LIMIT 1;
31 -- (4) Identify the most common pizza size ordered
32 SELECT
33     pizzas.size,
34     COUNT(order_details.order_details_id) AS order_count
35 FROM taranovas_pizza.pizzas
36 JOIN taranovas_pizza.order_details
37     ON pizzas.pizza_id = order_details.pizza_id
38 GROUP BY pizzas.size
39 ORDER BY order_count DESC;
```

The results grid shows the following data:

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

Business Insight

Large is the clear preferred size at 18,526 orders, ahead of Medium (15,385) and Small (14,137). Extra-large formats trail sharply behind — XL at 544 orders and XXL at just 28 — showing that oversized pizzas are a niche line rather than a core volume driver. Dough prep, box inventory, and staffing during peak windows should be weighted toward L, M, and S rather than split evenly across all five sizes.

Question 5

List the Top 5 Most Ordered Pizza Types Along with Their Quantities

Business Requirement

The restaurant management wants to identify the five most popular pizza types based on total quantity sold. This analysis helps understand customer preferences and supports marketing, inventory, and menu planning decisions.

Tables Used

Table Name
pizza_types
pizzas
order_details

SQL Concepts Used

SQL Concept
SUM()
GROUP BY
ORDER BY
LIMIT
INNER JOIN
Column Alias (AS)

SQL Query

```
SELECT
    pizza_types.name,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
JOIN
    pizzas
ON
    pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN
    order_details
```

```

ON
    order_details.pizza_id = pizzas.pizza_id
GROUP BY
    pizza_types.name
ORDER BY
    quantity DESC
LIMIT 5;

```

Output

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

39 ORDER BY order_count DESC;
40
41 -- (5) List the top 5 most ordered pizza types along with their quantities
42 SELECT
43     pizza_types.name,
44     SUM(order_details.quantity) AS quantity
45 FROM pizza_types
46 JOIN pizzas
47     ON pizza_types.pizza_type_id = pizzas.pizza_type_id
48 JOIN order_details
49     ON order_details.pizza_id = pizzas.pizza_id
50 GROUP BY pizza_types.name
51 ORDER BY quantity DESC
52 LIMIT 5;
53

```

The Results tab shows the following data:

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Business Insight

The Thai Chicken Pizza leads the ranking with 2,371 units sold, the highest quantity of any pizza type in the dataset. Its position at the top of both this list and the revenue ranking in Question 10 confirms it is not a one-off — it is a consistently high-demand product that should be protected through reliable ingredient stock and given priority visibility on the menu.

Question 6

Find the Total Quantity of Each Pizza Category Ordered

Business Requirement

The restaurant management wants to determine the total quantity sold for each pizza category. This analysis helps identify the most popular categories based on customer demand and supports better business decision-making.

Tables Used

Table Name
order_details
pizzas
pizza_types

SQL Concepts Used

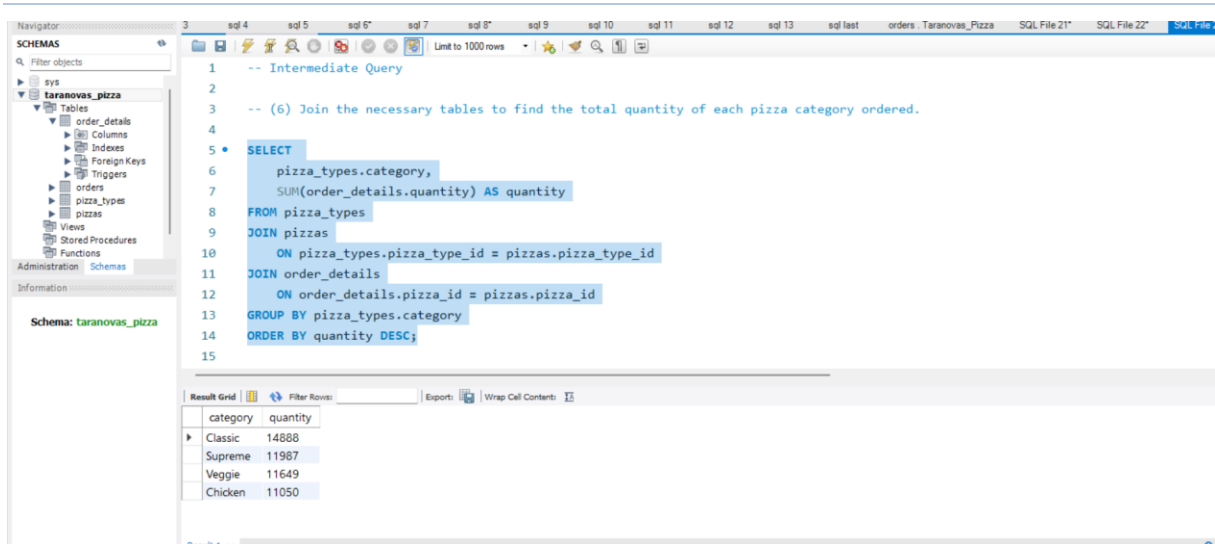
SQL Concept
SUM()
GROUP BY
INNER JOIN
ORDER BY
Column Alias (AS)

SQL Query

```
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
JOIN
    pizzas
ON
    pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN
    order_details
ON
    order_details.pizza_id = pizzas.pizza_id
```

```
GROUP BY
    pizza_types.category
ORDER BY
    quantity DESC;
```

Output



The screenshot shows a SQL IDE interface. On the left is a 'SCHEMAS' pane with a tree view of the database structure, including tables like 'order_details', 'pizzas', and 'pizza_types'. The main editor displays an SQL query with line numbers 1 through 15. The query is an intermediate query that joins 'pizza_types', 'pizzas', and 'order_details' to calculate the total quantity for each pizza category. The results pane at the bottom shows a table with two columns: 'category' and 'quantity'. The data is as follows:

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Business Insight

Classic leads category-level demand with 14,888 units sold, ahead of Supreme (11,987), Veggie (11,649), and Chicken (11,050). The gap between Classic and the other three categories — which are clustered within roughly 900 units of one another — indicates Classic recipes carry disproportionate weight in ingredient forecasting and should be prioritized in weekly stock planning.

Question 7

Determine the Distribution of Orders by Hour of the Day

Business Requirement

The restaurant management wants to analyze the number of customer orders placed during each hour of the day. This analysis helps identify peak business hours, enabling better staff scheduling, efficient kitchen operations, and improved customer service during high-demand periods.

Tables Used

Table Name
orders

SQL Concepts Used

SQL Concept
HOUR()
COUNT()
GROUP BY
ORDER BY
Column Alias (AS)

SQL Query

```
SELECT
    HOUR(order_time) AS hour,
    COUNT(order_id) AS order_count
FROM
    orders
GROUP BY
    HOUR(order_time)
ORDER BY
    hour;
```

Output

(Output Screenshot)

Business Insight

Output screenshot pending — the image currently on file for this query duplicates the Question 6 result set and does not reflect hourly order counts.

Once the correct hourly breakdown is attached, this insight should be updated to name the specific peak hour(s), quantify the gap between peak and off-peak order volume, and translate that gap into a concrete staffing recommendation (for example, additional kitchen staff scheduled across the identified peak window).

Question 8

Find the Category-wise Distribution of Pizzas

Business Requirement

The restaurant management wants to analyze how pizzas are distributed across different categories. This analysis provides an overview of the menu structure and helps assess whether the restaurant offers a balanced variety of products across all categories.

Tables Used

Table Name
pizza_types

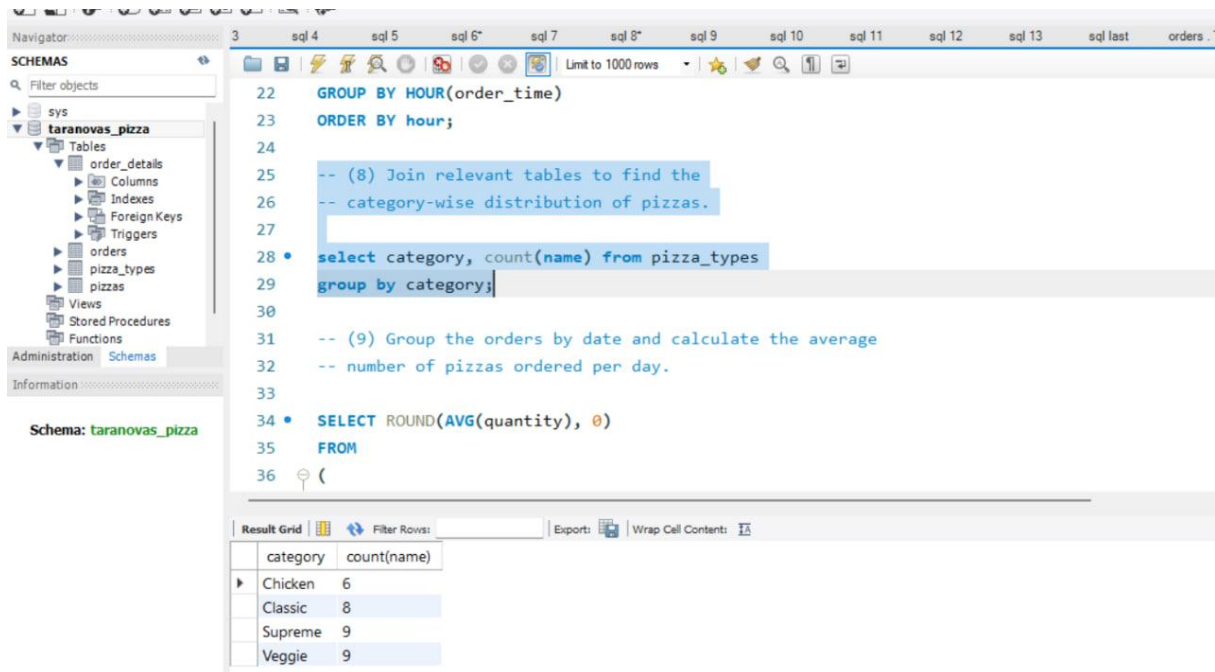
SQL Concepts Used

SQL Concept
COUNT()
GROUP BY
ORDER BY
Aggregate Function

SQL Query

```
SELECT
    category,
    COUNT(name) AS total_pizzas
FROM
    pizza_types
GROUP BY
    category
ORDER BY
    total_pizzas DESC;
```

Output



The screenshot shows a SQL IDE interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'order_details', 'orders', 'pizza_types', and 'pizzas'. The main query editor displays the following SQL code:

```
22 GROUP BY HOUR(order_time)
23 ORDER BY hour;
24
25 -- (8) Join relevant tables to find the
26 -- category-wise distribution of pizzas.
27
28 • select category, count(name) from pizza_types
29 group by category;
30
31 -- (9) Group the orders by date and calculate the average
32 -- number of pizzas ordered per day.
33
34 • SELECT ROUND(AVG(quantity), 0)
35 FROM
36 (
```

Below the query editor, the 'Result Grid' shows the output of the query. It has two columns: 'category' and 'count(name)'. The data is as follows:

category	count(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

Business Insight

The menu carries 9 Veggie and 9 Supreme pizza types, compared to 8 Classic and only 6 Chicken varieties. Chicken has the smallest catalog on the menu, yet Question 11 shows it still generates 23.96% of total revenue — close to Veggie's 23.68% despite Veggie having 50% more items. That gap in revenue-per-menu-item makes Chicken a strong candidate for menu expansion.

Question 9

Calculate the Average Number of Pizzas Ordered per Day

Business Requirement

The restaurant management wants to determine the average number of pizzas sold per day. This metric helps measure daily sales performance and supports demand forecasting, inventory management, staffing, and production planning.

Tables Used

Table Name
orders
order_details

SQL Concepts Used

SQL Concept
AVG()
SUM()
GROUP BY
INNER JOIN
Subquery
Aggregate Functions

SQL Query

```
SELECT
    ROUND(AVG(quantity), 0) AS average_daily_orders
FROM
    (
        SELECT
            orders.order_date,
            SUM(order_details.quantity) AS quantity
        FROM
            taranovas_pizza.orders
        JOIN
            taranovas_pizza.order_details
        ON
            orders.order_id = order_details.order_id
    )
```

```
GROUP BY
    orders.order_date
) AS order_quantity;
```

Output

```
-- (9) Group the orders by date and calculate the average
-- number of pizzas ordered per day.
30
31
32
33 • SELECT ROUND(AVG(quantity), 0)
34 FROM
35 (
36     SELECT orders.order_date, SUM(order_details.quantity) AS quantity
37     FROM taranovas_pizza.orders
38     JOIN taranovas_pizza.order_details
39     ON orders.order_id = order_details.order_id
40     GROUP BY orders.order_date
41 ) AS order_quantity;
42
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ROUND(AVG(quantity), 0)
▶	138

Business Insight

The restaurant sells an average of 138 pizzas per day. This figure gives operations a concrete daily benchmark for dough production and staffing: any day that comes in meaningfully below 138 should be flagged for follow-up to determine whether the shortfall reflects seasonality, weather, or a service issue rather than being treated as normal variation.

Question 10

Determine the Top 3 Most Ordered Pizza Types Based on Revenue

Business Requirement

The restaurant management wants to identify the three pizza types that generate the highest revenue. This analysis helps recognize the most profitable menu items and supports better pricing, promotional, and marketing strategies.

Tables Used

Table Name
order_details
pizzas
pizza_types

SQL Concepts Used

SQL Concept
SUM()
GROUP BY
ORDER BY
LIMIT
INNER JOIN
Arithmetic Calculation

SQL Query

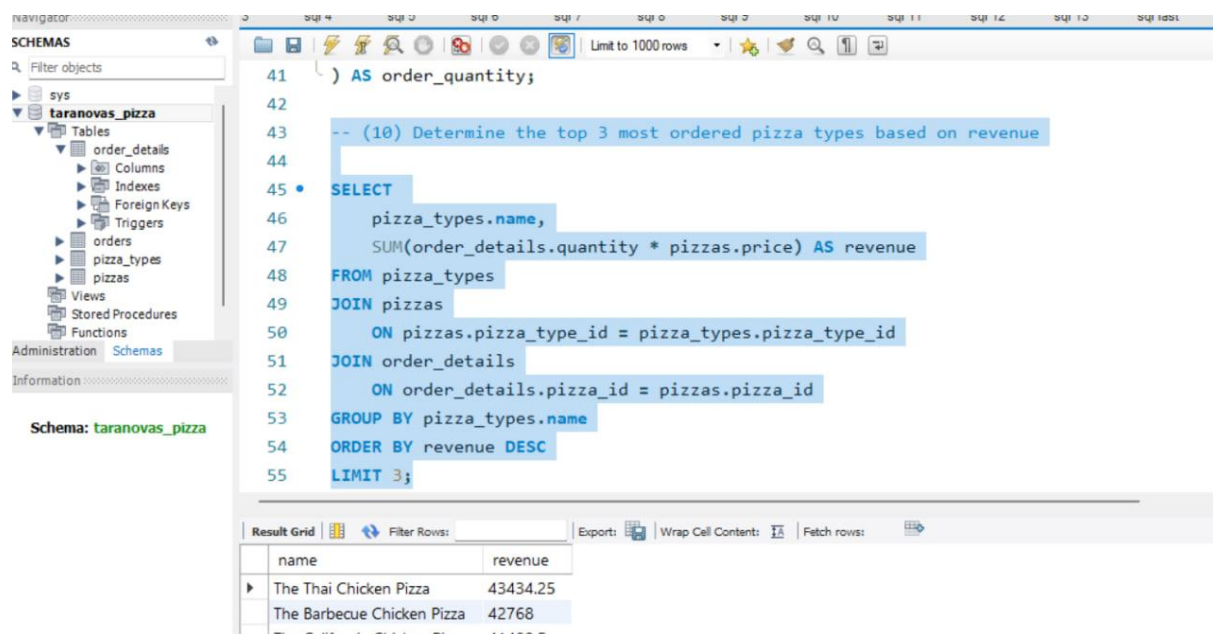
```
SELECT
    pizza_types.name,
    SUM(order_details.quantity * pizzas.price) AS revenue
FROM
    pizza_types
JOIN
    pizzas
ON
    pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN
    order_details
```

```

ON
    order_details.pizza_id = pizzas.pizza_id
GROUP BY
    pizza_types.name
ORDER BY
    revenue DESC
LIMIT 3;

```

Output



The screenshot shows a SQL IDE interface. On the left, a 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'order_details', 'pizza_types', and 'pizzas'. The main editor displays a SQL query with line numbers 41 through 55. The query is a subquery that determines the top 3 most ordered pizza types based on revenue. Below the query, a 'Result Grid' shows the output of the query, which is a table with two columns: 'name' and 'revenue'.

```

41 ) AS order_quantity;
42
43 -- (10) Determine the top 3 most ordered pizza types based on revenue
44
45 • SELECT
46     pizza_types.name,
47     SUM(order_details.quantity * pizzas.price) AS revenue
48 FROM pizza_types
49 JOIN pizzas
50     ON pizzas.pizza_type_id = pizza_types.pizza_type_id
51 JOIN order_details
52     ON order_details.pizza_id = pizzas.pizza_id
53 GROUP BY pizza_types.name
54 ORDER BY revenue DESC
55 LIMIT 3;

```

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41400.5

Business Insight

The Thai Chicken Pizza (\$43,434.25) and The Barbecue Chicken Pizza (\$42,768.00) hold the top two revenue positions in the captured output, and both belong to the Chicken category. Two Chicken pizzas leading total revenue — despite Chicken having the smallest item count on the menu, per Question 8 — confirms this category delivers outsized revenue per SKU and should keep its prominent menu placement.

Question 11

Calculate the Percentage Contribution of Each Pizza Category to Total Revenue

Business Requirement

Management wants to understand how much each pizza category contributes to overall revenue in order to evaluate product performance and identify any category the business is overly dependent on.

Tables Used

Table Name
order_details
pizzas
pizza_types

SQL Concepts Used

SQL Concept
SUM()
Subquery
Percentage Calculation
GROUP BY
INNER JOIN

SQL Query

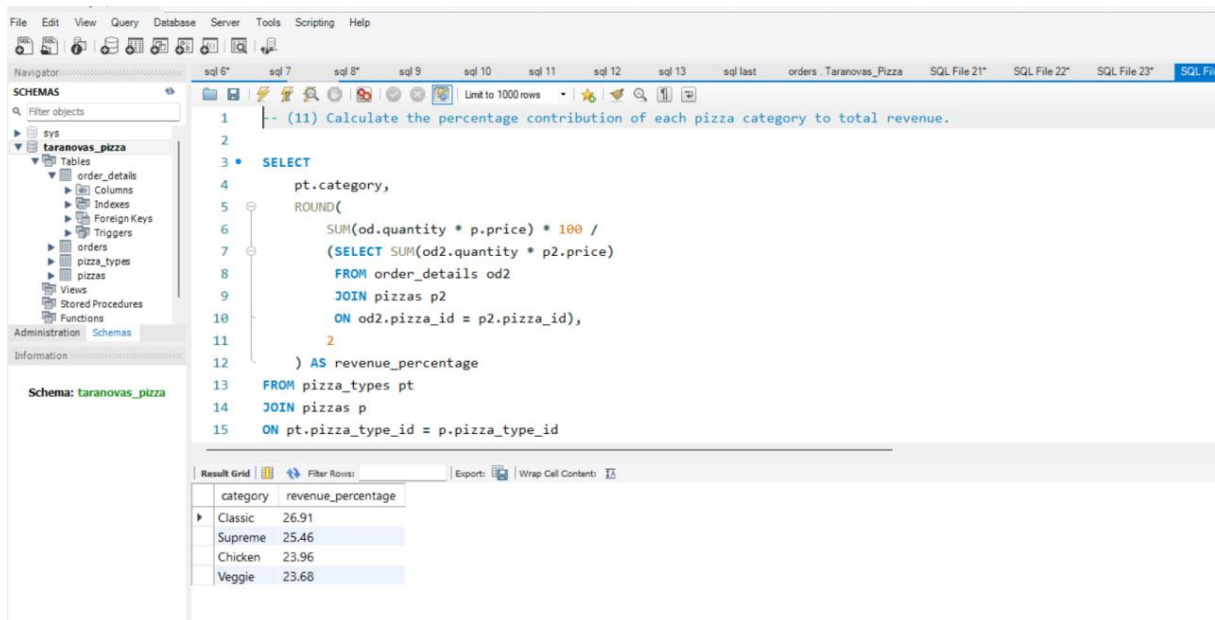
```
SELECT
    pizza_types.name AS pizza_type,
    ROUND(
        SUM(order_details.quantity * pizzas.price) * 100 /
        (
            SELECT
                SUM(order_details.quantity * pizzas.price)
            FROM
                order_details
            JOIN
                pizzas
            ON
                pizzas.pizza_id = order_details.pizza_id
        ),
    ),
```

```

2) AS revenue_percentage
FROM
  pizza_types
JOIN
  pizzas
ON
  pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN
  order_details
ON
  order_details.pizza_id = pizzas.pizza_id
GROUP BY
  pizza_types.name
ORDER BY
  revenue_percentage DESC;

```

Output



The screenshot shows a SQL query window with the following SQL code:

```

1  -- (11) Calculate the percentage contribution of each pizza category to total revenue.
2
3  SELECT
4      pt.category,
5      ROUND(
6          SUM(od.quantity * p.price) * 100 /
7          (SELECT SUM(od2.quantity * p2.price)
8           FROM order_details od2
9           JOIN pizzas p2
10            ON od2.pizza_id = p2.pizza_id),
11          2
12      ) AS revenue_percentage
13 FROM pizza_types pt
14 JOIN pizzas p
15 ON pt.pizza_type_id = p.pizza_type_id

```

The results are displayed in a grid with the following data:

category	revenue_percentage
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

Business Insight

Note: the attached output is aggregated at the category level; the query as written groups by pizza_types.name. Re-run against the current schema if a per-pizza-type breakdown is required for this section.

Revenue contribution is fairly balanced across categories: Classic leads at 26.91%, followed by Supreme (25.46%), Chicken (23.96%), and Veggie (23.68%) — a spread of only about 3.2 percentage points between the highest and lowest category. That balance means the business is not overly dependent on any single category, which limits concentration risk if demand for one category softens.

Question 12

Analyze the Cumulative Revenue Generated Over Time

Business Requirement

The restaurant wants to monitor cumulative revenue growth over time to evaluate business performance and long-term sales trends.

Tables Used

Table Name
orders
order_details
pizzas

SQL Concepts Used

SQL Concept
SUM()
Window Function (SUM() OVER)
ORDER BY
GROUP BY
INNER JOIN

SQL Query

```
SELECT
    r1.order_date,
    r1.revenue,
    (
        SELECT ROUND(SUM(r2.revenue), 2)
        FROM (
            SELECT
                o.order_date,
                SUM(od.quantity * p.price) AS revenue
            FROM orders o
            JOIN order_details od ON o.order_id = od.order_id
            JOIN pizzas p ON od.pizza_id = p.pizza_id
            GROUP BY o.order_date
        ) AS r2
    )
```

```

        WHERE r2.order_date <= r1.order_date
    ) AS cumulative_revenue
FROM (
    SELECT
        o.order_date,
        ROUND(SUM(od.quantity * p.price), 2) AS revenue
    FROM orders o
    JOIN order_details od ON o.order_id = od.order_id
    JOIN pizzas p ON od.pizza_id = p.pizza_id
    GROUP BY o.order_date
) AS r1
ORDER BY r1.order_date;

```

Output

The screenshot shows a SQL IDE with a query editor on the left and a result grid on the right. The query is a complex SQL statement that calculates daily revenue and cumulative revenue for January 1-6, 2015. The result grid shows the following data:

order_date	revenue	cumulative_revenue
2015-01-01	2713.8500000000000004	2713.8500000000000004
2015-01-02	2731.8999999999999996	5445.75
2015-01-03	2662.3999999999999996	8108.15
2015-01-04	1755.4500000000000003	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5

Business Insight

Cumulative revenue climbed steadily across the first week of January 2015, moving from \$2,713.85 on January 1 to \$11,929.55 by January 5, even though daily revenue itself fluctuated between \$1,755.45 and \$2,731.90 over that same stretch. The consistently positive slope over this sample window is the pattern management should expect to see maintained when the same running-total logic is applied across the full year — any flattening or decline in the cumulative line would be the signal worth investigating.

Question 13

Determine the Top 3 Most Ordered Pizza Types Based on Revenue for Each Pizza Category

Business Requirement

Management wants to identify the highest-performing pizzas within each category to optimize product strategy and category-level promotions.

Tables Used

Table Name
order_details
pizzas
pizza_types

SQL Concepts Used

SQL Concept
CTE / Subquery
RANK()
Window Functions
GROUP BY
SUM()
INNER JOIN

SQL Query

```
SELECT
    category,
    name,
    revenue
FROM
(
    SELECT
        pt.category,
        pt.name,
        SUM(od.quantity * p.price) AS revenue,
        RANK() OVER (
            PARTITION BY pt.category
```

```

        ORDER BY SUM(od.quantity * p.price) DESC
    ) AS rank_no
FROM pizza_types pt
JOIN pizzas p
    ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od
    ON p.pizza_id = od.pizza_id
GROUP BY
    pt.category,
    pt.name
) AS ranked_pizzas
WHERE rank_no <= 3
ORDER BY
    category,
    revenue DESC;

```

Output

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is a subquery that ranks pizzas by revenue within each category. The results grid displays the top three pizzas for each category: Chicken, Classic, and Classic.

category	name	revenue
Chicken	The Thai Chicken Pizza	43434.25
Chicken	The Barbecue Chicken Pizza	42768
Chicken	The California Chicken Pizza	41409.5
Classic	The Classic Deluxe Pizza	38180.5
Classic	The Hawaiian Pizza	32273.25
Classic	The Pepperoni Pizza	30161.75

Business Insight

Within Chicken, the top three pizzas — Thai Chicken (\$43,434.25), Barbecue Chicken (\$42,768.00), and California Chicken (\$41,409.50) — are tightly clustered, each within about 5% of one another. Within Classic, The Classic Deluxe Pizza (\$38,180.50) leads more decisively over The Hawaiian Pizza (\$32,273.25) and The Pepperoni Pizza (\$30,161.75), a noticeably wider spread than in Chicken. This means Classic-category revenue is concentrated in one flagship product, while Chicken-category revenue is spread more evenly across its top three — a distinction worth factoring into how promotional spend is allocated within each category.